

Task-Oriented Robotic Grasping and Manipulation using Reinforcement Learning

Aiden Slabiak

CS-484: Reinforcement Learning

The University of Alabama

Tuscaloosa, AL, USA

ajslabiak@crimson.ua.edu

Abstract—This project investigates the application of Reinforcement Learning (RL) to solve robotic manipulation problems, specifically focusing on the “lift” skill for a dispenser (Object 3686). Unlike general grasping, task-oriented affordance requires the robot to understand the functional relationship between an object’s physical properties and the requirements of a specific goal. Using the AffordPose database to define initial hand postures and the StableBaselines3 library for algorithm implementation, I trained a robotic agent to execute a lift operation. The results evaluate the effectiveness of the Proximal Policy Optimization (PPO) algorithm and the impact of custom reward functions on task success rates.

Index Terms—Reinforcement Learning, PPO, Robotics.

I. INTRODUCTION

Robotic manipulation is a fundamental problem in robotics, with applications ranging from industrial automation to assistive technologies and autonomous systems. While modern robotic systems have achieved a strong performance in structured environments, manipulating objects in unstructured, and random real world settings remains a significant challenge. This difficulty arises from the need to handle complex object shapes, uncertain contact dynamics, and the requirement for precise coordination between perception and control.

Traditional approaches to robotic grasping often rely on geometric reasoning and analytical models, where objects are simplified into basic shapes and grasp stability is evaluated using metrics such as force closure. Although this proves to be effective for basic pickup and place tasks, these methods struggle to generalize to more complex interactions where the way an object is grasped directly affects the success of a task later on. As a result, learning-based approaches like RL, can enable agents to learn manipulation strategies through interaction with their environment.

It is also true that standard RL approaches introduce their own challenges. In larger spaces, agents must explore a very large number of possible actions before discovering meaningful contact with an object. This often leads to inefficient learning, where a large portion of training time is spent on trivial behaviors such as reaching or accidental collisions rather than meaningful task progress.

Robotic manipulation in realistic environments therefore requires more than just simple geometric reaching. It requires an understanding of how an object should be engaged with to perform a task. This idea is captured by task-oriented

object affordance, which describes the relationship between an object’s physical properties and its suitability for specific actions.

For a dispenser (Object 3686), a “lift” operation is not just about making contact and raising the object. It requires an appropriate hand posture and force distribution that preserves both the agent’s and the object’s orientation throughout the whole task.

A. Problem Discussion

A traditional robotic grasping project might treat objects as generic geometries to be held and lifted. In contrast, task-oriented manipulation emphasizes that grasp strategies should be driven by the intended use of the object. For example, a robot might grasp a cup differently depending on the task. Holding the cup around the body when drinking, but gripping it by the base when placing it into a dishwasher. In this project, the goal is to guide a robotic hand from a predefined initial position, through a coordinated sequence of actions that enables it to securely grasp and lift a dispenser. The core challenge lies in defining effective reward functions and termination conditions that enable the stable and successful training. This requires not only careful tuning of action incentives, but also an understanding of how rewards shape and influence the agent’s behavior over time.

B. Summary of Procedure and Method

The methodology employed in this project follows a simple RL pipeline described below:

- **Environment Setup:** A virtual simulation was configured using the PyBullet physics engine, with library dependencies managed via a standard Python environment. The workspace, illustrated in Figure 1, includes the robotic hand model and the target object.
- **Affordance Integration:** The robotic hand was initialized using a specific posture mapped to the dispenser (Object 3686) from the AffordPose database. As shown in the simulation environment (Figure 1), this ensures the starting state is functionally relevant to the task of grasping and lifting.
- **Algorithm Implementation:** The PPO algorithm, a modern on-policy RL method, was selected for its stability

in continuous action spaces, allowing the hand to learn smooth movements.

- **Training and Evaluation:** The agent was trained over a series of episodes using a custom reward function designed to encourage contact and lift height. Following training, we used an `evaluate.py` script to measure success rates and learning efficiency under various starting conditions.

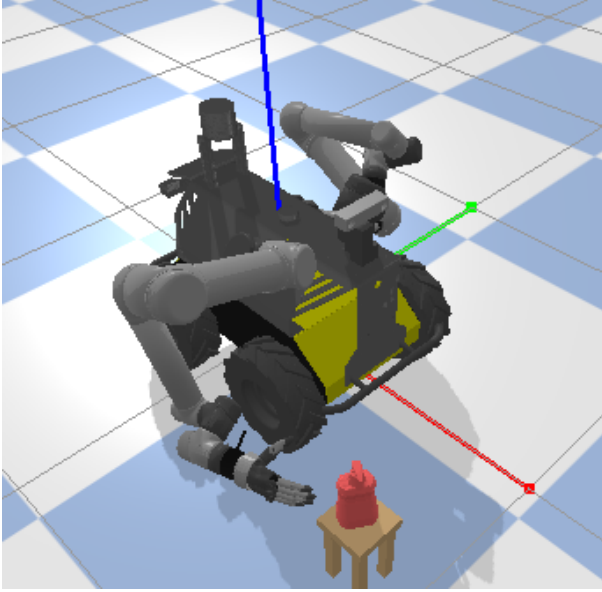


Fig. 1. The PyBullet simulation environment featuring the multi-fingered robotic hand initialized with an affordance pose and Object 3686 (Dispenser).

II. PROBLEM FORMULATION

In this project, the objective is to learn a policy that transitions a robotic hand from a *reach* state into a *manipulation* state. This transition is difficult because the robot must move from having no contact with the dispenser to a coordinated, stable grasp capable of lifting the object.

We frame this as a RL problem. In RL, the robot learns by trial and error, aiming to maximize its long-term success score, formally known as the **expected discounted return**:

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad (1)$$

We can break this equation down into three critical parts:

- **The Total Reward** ($\sum r_t$): This is the sum of every small point the robot earns during an attempt, such as points for getting closer to the object, points for making contact, and points for lifting it successfully.
- **The Discount Factor** (γ^t): This is a value between 0 and 1 that tells the model the importance of a reward. It tells the robot that rewards earned *sooner* are slightly more valuable than rewards earned *later*. This encourages the robot to be efficient.

- **The Expectation ($\mathbb{E}$):** Since the environment is complex and slightly unpredictable, the robot cannot guarantee the exact same score every time. The $\mathbb{E}$ symbol means the robot is trying to maximize its *average* score over thousands of different attempts.

By maximizing this value, the policy π_{θ} learns to balance the transition from reaching to grasping. It discovers that it cannot get the big lifting reward at the end unless it first carefully coordinates its finger placement and ensures not to drop the object in the manipulation phase.

III. BACKGROUND AND RELATED WORK

The evolution of robotic grasping has transitioned from low dimension modeling of simple shapes to flexible, data-driven RL agents. This section explains the move from geometric stability to task-oriented learning.

Early work in robotic manipulation relied on analytical methods, where grasp success was determined by how well the hand could hold an object based on contact forces. Ideas like *force closure* and *form closure* were used to check whether a grasp could resist disturbances, using friction and object shape. While these methods are effective in very controlled settings, they often require accurate 3D models and precise sensing. This makes them difficult to apply in real-world environments, where object shape may be uncertain.

To overcome the limitations of these hard-coded solutions, RL has emerged as a modern framework for robotic manipulation strategies. Deep RL, specifically, allows agents to map sensory inputs such as joint states and object positions, to lower level motor actions that control the robot's joints.

PPO has become a benchmark algorithm for these tasks due to its stable update mechanism. It improves the policy gradually instead of making large sudden changes, which helps prevent training from becoming unstable. This is especially important in environments like PyBullet, where small changes in how the robot touches an object can lead to large and unpredictable changes in the reward signal, making learning difficult if updates are not carefully controlled.

Overall, the topic has moved from purely analytical grasping toward integrated systems that combine learning and prior knowledge to accurately simulate robotic grasping. This project builds on these ideas by leveraging task-informed initialization and reinforcement learning to address the challenges of stable manipulation.

IV. METHODOLOGY

This section details the implemented process of the dispenser manipulation task as a Reinforcement Learning problem, the underlying theory, and the specific design of the environment.

A. Reinforcement Learning Framework

We model the task as a Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. In this formulation, the agent represents the robotic hand, and it learns by repeatedly interacting with the PyBullet environment over discrete timesteps t .

At each timestep, the agent observes the current state $s_t \in \mathcal{S}$, which includes information about the robot and object configuration. Based on this observation, it selects an action $a_t \in \mathcal{A}$ that controls the robot's movements. After executing the action, the environment returns a reward r_t , which measures how good the action was for completing the task, and the system transitions to a new state s_{t+1} .

B. Proximal Policy Optimization

To solve the MDP, we utilize **PPO**. In robotics, learning is often unstable so a single wrong large update to the robot's control logic can cause the agent loses progress and fail to perform basic movements. PPO addresses this by ensuring that the robot's behavior improves through small, cautious steps rather than erratic shifts.

1) *The Probability Ratio*: To quantify how much the robot's policy changes during an update, we define a probability ratio, $r_t(\theta)$. This compares the likelihood of an action under the new policy to the likelihood of that same action under the old policy:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \quad (2)$$

If $r_t(\theta) = 1$, the robot is behaving exactly as it did before. If the ratio is greater than 1, the action has become more likely in the new version.

2) *The Clipped Objective*: The advantage of PPO is the **Clipped Surrogate Objective**. It acts as a safety net, preventing the robot from changing its behavior too drastically based on a single piece of feedback. The objective function is defined as:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (3)$$

To understand this consider the three main components:

- **The Advantage ($\hat{A}_t$)**: This is a score indicating if an action was "better than average." If $\hat{A}_t$ is positive, the robot wants to do that action more often.
- **The Clip**: Even if an action resulted in a huge reward, the `clip` function prevents the robot from increasing the probability of that action by more than a small margin (controlled by ϵ , typically set to 0.2 or 20%).
- **The Minimum (min)**: This ensures we take the more conservative estimate of the improvement, prioritizing stability over rapid and risky learning.

3) *Actor-Critic Architecture*: As illustrated in Figure 2, PPO is implemented as an **Actor-Critic** model. This creates an interesting dynamic balancing power:

- 1) **The Actor**: The network that decides which joint torques to apply to the robotic hand.
- 2) **The Critic**: A secondary network that predicts the expected reward from the current state.

By comparing the Actor's actual results to the Critic's predictions, the system calculates the Advantage ($\hat{A}_t$). This allows the robot to learn not just from success, but from how much

better a specific move was compared to what it expected to happen.

Fig. 2. The PPO Actor-Critic architecture. The Actor network outputs continuous control signals for the joints, while the Critic network evaluates the state to provide stable feedback for updates.

C. Neural Network Architecture and Implementation

The learning agent in this project utilizes a Multi-Layer Perceptron (MLP) architecture, implemented via the Stable-Baselines3 library. Both networks consist of two hidden layers with 64 neurons each, utilizing Hyperbolic Tangent activation functions. This configuration provides a balanced representational capacity for modeling the non-linear contact dynamics required for object manipulation while maintaining computational efficiency.

To ensure stability, the project utilizes `VecNormalize` to perform normalization of observations and rewards. Raw observations and reward signals can differ by several orders of magnitude. Without normalization, these disparities can lead to vanishing or exploding numbers, preventing the agent from converging on a stable lift policy.

D. Hyperparameter Configuration

The performance of PPO is highly sensitive to its hyperparameter configuration. Through iterative testing, a low learning rate was found to be essential for maintaining the benefits of the task-informed initialization. If the learning rate is too high, the agent quickly deviates from the AffordPose posture before discovering stable contact. The final parameters used for the 5.12×10^6 maximum timestep training run are detailed in Table II.

TABLE I
PPO TRAINING HYPERPARAMETERS FOR DISPENSER LIFT

Hyperparameter	Value	Rationale
Learning Rate	3×10^{-5}	Ensures stable policy updates
Batch Size	32	Balances throughput and accuracy
Gamma (γ)	0.99	Prioritizes long-term task success
Clip Range (ϵ)	0.2	Prevents large policy shifts
n_steps	2048	Default for PPO stability

E. Formal State and Action Space

The observation space $\mathcal{S}$ consists of many features that describe both the robot configuration and its interaction with the object. Rather than relying on raw sensor inputs alone, the state encodes relative geometric and contact information gathered from the simulation.

The observation at time t includes:

- Relative geometric relationships between the hand and object
- Finger specific spatial features gathered from fingertip interactions
- Contact signals such as contact activation and stability indicators

- Distance and alignment features between the end-effector and the target grasp region
- Force and torque based estimates of object stability

This representation is designed to provide sufficient information for both reaching and stable grasp execution.

F. Reward Design and Shaping

The reward function is formulated as a composite objective function, transitioning between a reaching phase (R_{s1}) and a manipulation phase (R_{s2}). The stage transition occurs dynamically when the end-effector enters a predefined spatial tolerance ϵ_{gl} around the target grasp pose.

$$R_{total} = \begin{cases} R_{s1}, & \text{if stage} = 1 \\ 80 + R_{s2}, & \text{if stage} = 2 \end{cases} \quad (4)$$

where the stage is determined dynamically based on proximity and grasp conditions.

1) *Stage 1: Reaching and Alignment*: In the reaching phase, the agent is incentivized to minimize the distance to the goal, align its approach vector, and maintain precision along individual Cartesian axes. The component R_{s1} is defined as:

$$R_{s1} = R_{dist} + 50 \cdot R_{dir} - R_{collision} + \sum_{i \in \{x, y, z\}} B_i \quad (5)$$

where:

- **Distance Reward**: $R_{dist} = \frac{1}{\|p_{hand} - p_{target}\| + 1}$ provides a continuous signal to pull the hand toward the object.
- **Directional Alignment**: Calculated as the cosine similarity between the initial and current direction vectors:

$$R_{dir} = \frac{\vec{v}_{init} \cdot \vec{v}_{curr}}{\|\vec{v}_{init}\| \|\vec{v}_{curr}\|} \quad (6)$$

- **Collision Penalty**: $R_{collision}$ is equivalent to the number of contact points detected between the robot and the object, penalizing premature or unstable contact.
- **Precision Bonuses**: B_i is a sparse bonus of +2 applied if the hand's position along axis i is within a threshold ϵ_{near} of the target.

2) *Stage 2: Stability and Lifting*: Upon successful positioning, the reward structure shifts to prioritize the physical security of the grasp and the vertical position of the object.

$$R_{s2} = R_{stability} + R_{lift} \quad (7)$$

The stability component $R_{stability}$ utilizes the physics engine to ensure a robust grasp:

$$R_{stability} = 2C + \frac{1}{\sum |\vec{F}| + 1} + \frac{1}{\sum |\vec{\tau}| + 1} \quad (8)$$

where C is the count of active contact points, and $\vec{F}$ and $\vec{\tau}$ are the net force and torque residuals acting on the object. This formulation encourages the agent to achieve a state of static equilibrium.

The lifting component R_{lift} provides a signal to incentivize upward motion:

$$R_{lift} = 5000 \cdot \max(0, z_{curr} - z_{init}) \quad (9)$$

where z_{init} is the initial object height. The large scaling factor ensures that successful lifting is a strong driver for policy optimization once a stable grasp is established.

3) *Termination Conditions*: The episode terminates if any of the following logical conditions are met:

$$T = \begin{cases} \text{Timeout,} & \text{if } t > t_{max} \\ \text{Out of Range,} & \text{if } p_{obj} \notin \text{Workspace} \\ \text{Grasp Failure,} & \text{if stage} = 2 \text{ and } \neg \text{isGrasped} \end{cases} \quad (10)$$

V. RESULTS

The trained agent was evaluated using the `evaluate.py` script over 643 independent runs to measure consistency and success rate. A successful episode was defined as the robot achieving a stable grasp on the dispenser followed by a sustained lift without slipping, tipping, or loss of contact.

The quantitative performance metrics are summarized in Table II. The data demonstrates that the task-informed initialization approach produced highly reliable results, achieving an average success rate of approximately 99%.

TABLE II
QUANTITATIVE PERFORMANCE SUMMARY FOR DISPENSER LIFT TASK

Metric	Value
Algorithm	Proximal Policy Optimization
Object ID	3686 (Dispenser)
Total Training Timesteps	5.12×10^6
Evaluation Episodes	643
Average Success Rate	99.69%
Mean Steps to Success	412
Training Stabilization	10–15 Episodes

Qualitatively, the learned policy demonstrated smooth coordination between arm positioning and finger closure. Early in training, failures were primarily caused by running out of time or failure to get within grasping distance, leading to object slippage during lifting. As training progressed, the agent learned to adjust its grasp more deliberately, aligning the fingers around sturdy regions of the dispenser before lifting.

One notable observation was the sensitivity of performance to small variations in reward tuning. Slight changes in penalty terms for instability or misalignment had a noticeable impact on whether the agent developed a stable lifting strategy or converged to partial, non-functional grasps. This reinforces the importance of carefully balancing reward components in RL manipulation tasks.

Additionally, PPO proved to be effective for this environment, successfully handling the continuous control space and converging to a stable policy without requiring extensive parameter tuning. However, the results also suggest that convergence quality is highly dependent on the quality of initialization and reward shaping.

Overall, the results demonstrate that combining task-aware initialization with reinforcement learning significantly improves both learning efficiency and final task performance in complex robotic manipulation scenarios.

VI. DETAILED PERFORMANCE ANALYSIS

To better understand the agent’s behavior, we performed a granular analysis of the reward trajectories and success criteria across a 643-episode evaluation window. The dataset revealed a success rate of 99.69%, with a mean episode reward of approximately 1.3×10^5 .

A. Reward Convergence and Stability

As shown in Figure 3, the agent’s reward exhibits a rapid ascent within the first 50,000 timesteps. This phase corresponds to the agent quickly mastering the transition from the reaching state to the manipulation state. Because the hand is initialized in a task-informed posture from the AffordPose database, the agent avoids the need to learn how to move fingers and joints to setup a grasp and begins establishing meaningful contact almost immediately.

After this initial surge, the reward stabilizes and then climbs again toward a plateau near 200,000 timesteps. The minor oscillations observed in these later stages represent the agent fine-tuning fingertip pressure to maintain the lift without triggering any penalties. This shows that while the major movements are learned within the first 100k steps, the more delicate force coordination required for a 99% reliability accounts for the remainder of the training duration.

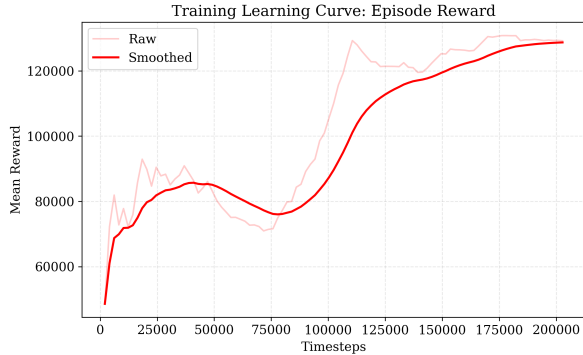


Fig. 3. Learning curve illustrating cumulative reward over all timesteps. The smoothed red line represents an exponential moving average (EMA) showing stable convergence.

B. Training Convergence Metrics

To confirm the reliability of the PPO algorithm, we monitored internal training metrics beyond simple reward totals.

- **Value Loss (Fig. 4):** The value loss represents the error in the Critic network’s predictions. After an initial spike during the exploration phase, the loss decays steadily toward zero. This shows that the Critic has successfully learned to predict the expected score of any given hand position.

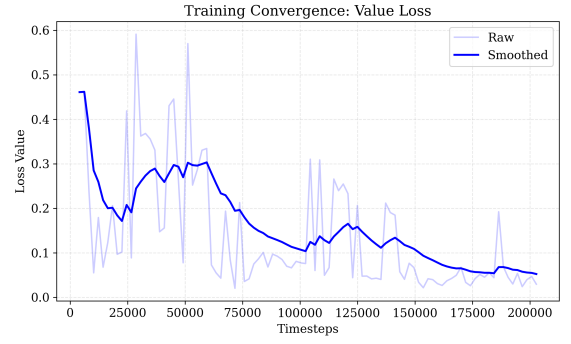


Fig. 4. Value loss convergence during the training process, showing the decay of error in reward estimation as the Critic network stabilized.

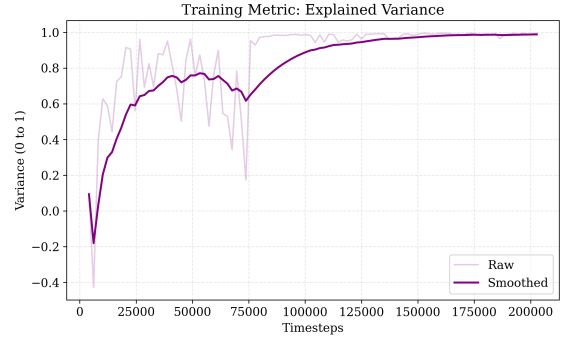


Fig. 5. Explained variance progression, reaching a terminal value of 0.99, confirming a high accuracy.

- **Explained Variance (Fig. 5):** This metric measures how well the Critic accounts for the variation in rewards. A value of 1.0 represents a perfect prediction. In our trials, the variance reached 0.99, confirming that the agent developed a near-perfect internal model of the task’s value. A distinct jump is visible at 75,000 timesteps, likely marking the exact moment the agent mastered the lift dynamics.

C. Failure Mode Analysis

Following training, the policy was frozen and tested over 643 independent evaluation episodes to measure its robustness. As illustrated in Figure 6, the model achieved a 99.69% success rate.

Despite this near perfect performance, two specific “out-of-range” failures occurred (0.31%). These failures were characterized by a severe reward penalty (approx. $-15,000$). Physical observation of the simulation suggests these failures occurred when excessive force was applied during the initial lift, causing the dispenser to pivot unexpectedly. The resulting instability likely triggered a feedback loop where the agent over compensated for the tilt, eventually pushing the hand outside the defined workspace boundaries. While statistically rare, these cases highlight the difficulty of mastering edge cases.

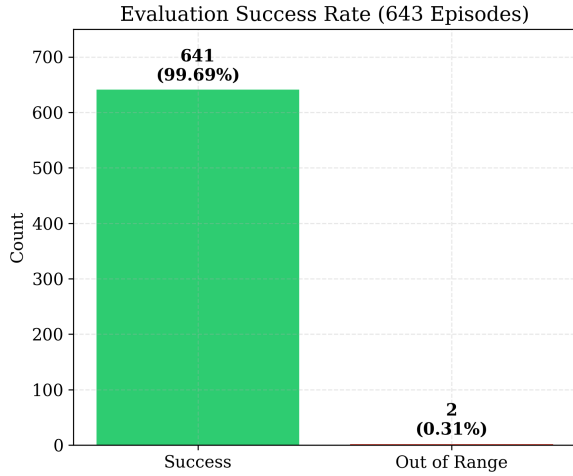


Fig. 6. Success vs. Failure distribution over evaluation episodes. The model achieved a nearly perfect 99.69% success rate.

VII. DISCUSSION

The results of this project show the importance of incorporating task knowledge into RL. While standard RL methods are capable of learning behaviors from scratch, performance is often limited by inefficient exploration in higher dimension action spaces.

A. Performance and Utility of PPO

The selection of PPO was a large factor in achieving the high reliability and 99.69% success rate achieved in this project.

- **Advantages:** PPO’s main strength in this environment is its stability. By utilizing the clipped surrogate objective, the algorithm acts as a safety net, preventing the robot from making unnecessarily large shifts in joint forces that could lead to failure during either of the described phases. This was particularly effective for handling the multi-fingered hand.
- **Disadvantages:** However, PPO proved highly sensitive to hyperparameter tuning and reward shaping. If the learning rate was set too high (above 3×10^{-5}), I noticed the agent might deviate from the AffordPose posture, losing the benefits of the previously mentioned task initialization before it could grasp.
- **Success Justification:** Ultimately, PPO was successful because of its cautious update mechanism allowing the agent to learn the tiny movements required for the task.

B. Analysis of the Reward Function

The designed reward function used a, two stage structure to guide the agent through the transition from reaching to lifting.

- **Key Factors:**
 - **Stage 1 (Reaching):** Factors like directional alignment (R_{dir}) and the collision penalty ($R_{collision}$) were essential. The collision penalty specifically

prevented the agent from crashing into the dispenser, ensuring it approached for a functional grasp.

- **Stage 2 (Stability & Lifting):** Once a grasp was detected, the lifting component (R_{lift}) was another useful reward mechanism. A large factor of 5000 was applied to ensure the agent prioritized upward motion over simply holding the object.

- **Effectiveness:** The two staged transition worked well, as shown by the rapid reward gain within the first 50,000 timesteps. However, the function was sensitive early on. Low factors in the lift reward (R_{lift}) led to policies that would achieve a grasp and continue to not lift so much.

Overall, the study suggests that task oriented design choices are key factors in enabling effective RL.

VIII. CONCLUSION

This project studied reinforcement learning for task-oriented robotic manipulation, focusing on a lift task involving a dispenser object. Unlike traditional grasping approaches that model objects as simple geometric shapes, this work emphasized the importance of task specific affordances in guiding behavior.

The main finding is that task aware initialization significantly improves learning efficiency and overall performance. Initializing the robotic hand using posture priors from the AffordPose dataset places the agent in a near ready grasp configuration, reducing the need for extensive exploration and allowing learning to focus on improving stability and control during object lifting.

Experiments demonstrate that combining PPO with affordance based initialization leads to stable and consistent manipulation policies in simulation. However, the results also show that performance is highly sensitive to reward design, particularly in how contact quality, stability, and lift success are balanced.

Overall, this work shows that performance in robotic manipulation is strongly influenced by how the problem is structured, not just the learning algorithm itself. Initialization strategy, reward shaping, and task design play a large role in determining success. Future work will explore more complex manipulation tasks and investigate the transfer for real-world deployment.

REFERENCES

- [1] J. Schulman et al., “Proximal Policy Optimization Algorithms,” 2017.
- [2] A. Raffin et al., “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” 2021.
- [3] E. Coumans and Y. Bai, “PyBullet, a Python module for physics simulation,” 2016.
- [4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [5] X. Zhang et al., “Actor-critic proximal policy optimization architecture,” 2023. [Online].
- [6] OpenAI, “Spinning Up in Deep RL: Proximal Policy Optimization,” [Online].
- [7] T. Simonini et al., “The Hugging Face Deep Reinforcement Learning Course: Unit 8 - Proximal Policy Optimization (PPO),” [Online].
- [8] K. Huang, “The 37 Implementation Details of Proximal Policy Optimization,” *CleanRL Blog*, 2022. [Online]. Available: <https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/>

- [9] L. Weng, "Policy Gradient Algorithms," *Lil'Log*, 2018. [Online]. Available: <https://lilianweng.github.io/posts/2018-04-08-policy-gradient/>
- [10] B. Rana, "Deep Reinforcement Learning: The Actor-Critic Method," *Towards Data Science*, 2026. [Online].